SIMATS ENGINEERING

ASSESSMENT TOOL 1-CO3

(Analytical Problem Solving)

COURSE CODE: ITA04
COURSE NAME: STATISTICS WITH R PROGRAMMING
COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 20 **Total Marks: 100**

ANNEXURE A

DESIGN TASK QUESTIONS

Code of Conduct:

I Dhanushree S/192421218 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Dhanushree S

DESIGN TASK QUESTIONS

1. Data Frames – Creation and Structure

A college wants to maintain student records containing Student ID, Name, Department, Marks, and Attendance. Analyze the suitability of a data frame for storing these records. Write an R program to create the data frame, display its structure, and identify the students who scored above 80 marks.

CODE:

```
students <- data.frame(
  Student_ID = c(101, 102, 103, 104, 105),
  Name = c("Arun", "Priya", "Kavi", "Ravi", "Meena"),
  Department = c("IT", "CSE", "IT", "ECE", "CSE"),
  Marks = c(85, 72, 91, 65, 88),
  Attendance = c(90, 85, 95, 70, 92)
)
print(students)
str(students)
high_scorers <- students[students$Marks > 80, ]
print(high_scorers)
```

OUTPUT:



```
students <- data.frame(
  Student_ID = c(101, 102, 103, 104, 105),
  Name = c("Arun", "Priya", "Kavi", "Ravi", "Meena"),
  Department = c("IT", "CSE", "IT", "ECE", "CSE"),
  Marks = c(85, 72, 91, 65, 88),
  Attendance = c(90, 85, 95, 70, 92)
)
print(students)
str(students)
high_scorers <- students[students$Marks > 80, ]
print(high_scorers)
```

```
STDIN
Input for the program ( Optional )

Output:
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    72         85
3        103  Kavi         IT    91         95
4        104  Ravi        ECE    65         70
5        105 Meena        CSE    88         92
'data.frame':   5 obs. of  5 variables:
 $ Student_ID: num  101 102 103 104 105
 $ Name      : chr  "Arun" "Priya" "Kavi" "Ravi" ...
 $ Department: chr  "IT" "CSE" "IT" "ECE" ...
 $ Marks     : num  85 72 91 65 88
 $ Attendance: num  90 85 95 70 92
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
3        103  Kavi         IT    91         95
5        105 Meena        CSE    88         92
```

2. Making Data Frames

A company maintains employee details, including Employee ID, Name, Age, Salary, and Department. Develop an R program to create a data frame using vectors. Analyze how R handles columns with different data types and explain the importance of maintaining equal-length vectors.

CODE:

```
emp_id <- c(201, 202, 203, 204)

emp_name <- c("Anu", "Bala", "Charan", "Divya")

age <- c(25, 30, 28, 26)

salary <- c(30000, 45000, 38000, 42000)

department <- c("IT", "HR", "Sales", "IT")

employees <- data.frame(

  Employee_ID = emp_id,

  Name = emp_name,
```

Age = age,

Salary = salary,

Department = department

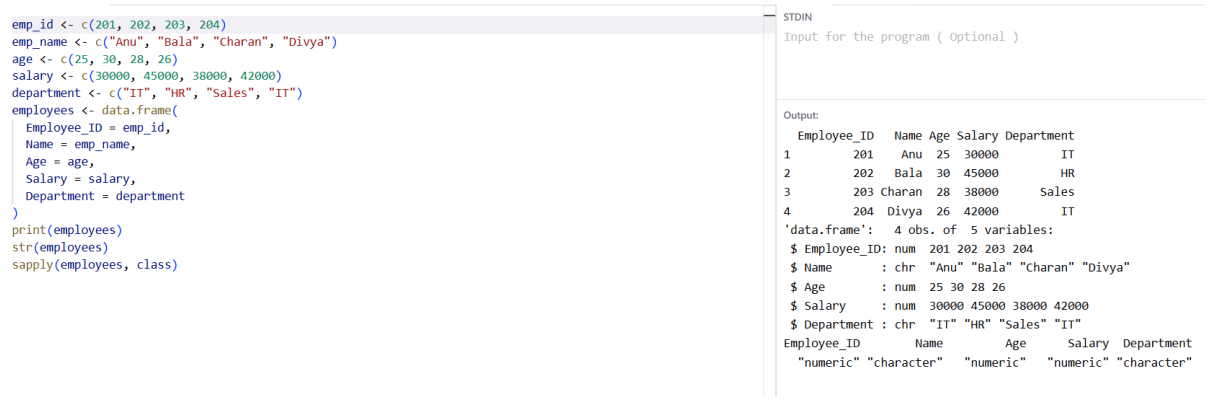
)

print(employees)

str(employees)

sapply(employees, class)

OUTPUT:



3. Working with Data Frames

Consider a data frame containing student names, marks in three subjects, and attendance percentages. Write an R program to calculate each student's total and average marks, identify students who passed all subjects, and display students with attendance above 75%. Analyze how row and column operations simplify data processing.

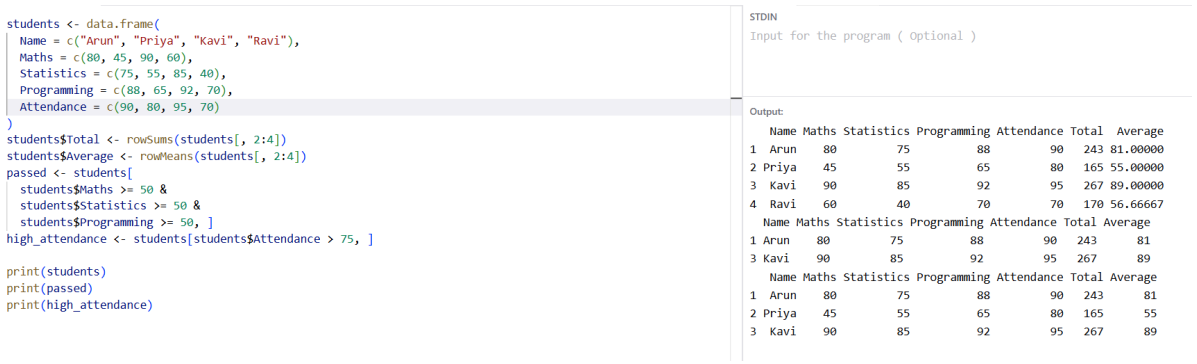
CODE:

students <- data.frame(

Name = c("Arun", "Priya", "Kavi", "Ravi"),

```
  Maths = c(80, 45, 90, 60),
  Statistics = c(75, 55, 85, 40),
  Programming = c(88, 65, 92, 70),
  Attendance = c(90, 80, 95, 70)
)
students$Total <- rowSums(students[, 2:4])
students$Average <- rowMeans(students[, 2:4])
passed <- students[
  students$Maths >= 50 &
  students$Statistics >= 50 &
  students$Programming >= 50, ]
high_attendance <- students[students$Attendance > 75, ]
print(students)
print(passed)
print(high_attendance)
```

OUTPUT:



```
students <- data.frame(
  Name = c("Arun", "Priya", "Kavi", "Ravi"),
  Maths = c(80, 45, 90, 60),
  Statistics = c(75, 55, 85, 40),
  Programming = c(88, 65, 92, 70),
  Attendance = c(90, 80, 95, 70)
)
students$Total <- rowSums(students[, 2:4])
students$Average <- rowMeans(students[, 2:4])
passed <- students[
  students$Maths >= 50 &
  students$Statistics >= 50 &
  students$Programming >= 50, ]
high_attendance <- students[students$Attendance > 75, ]

print(students)
print(passed)
print(high_attendance)
```

STDIN

Input for the program (Optional)

Output:

```
   Name Maths Statistics Programming Attendance Total  Average
1  Arun    80         75          88         90   243 81.00000
2 Priya    45         55          65         80   165 55.00000
3  Kavi    90         85          92         95   267 89.00000
4  Ravi    60         40          70         70   170 56.66667
  Name Maths Statistics Programming Attendance Total Average
1 Arun    80         75          88         90   243      81
3 Kavi    90         85          92         95   267      89
   Name Maths Statistics Programming Attendance Total Average
1  Arun    80         75          88         90   243      81
2 Priya    45         55          65         80   165      55
3  Kavi    90         85          92         95   267      89
```

4. Adding, Modifying, and Deleting Data

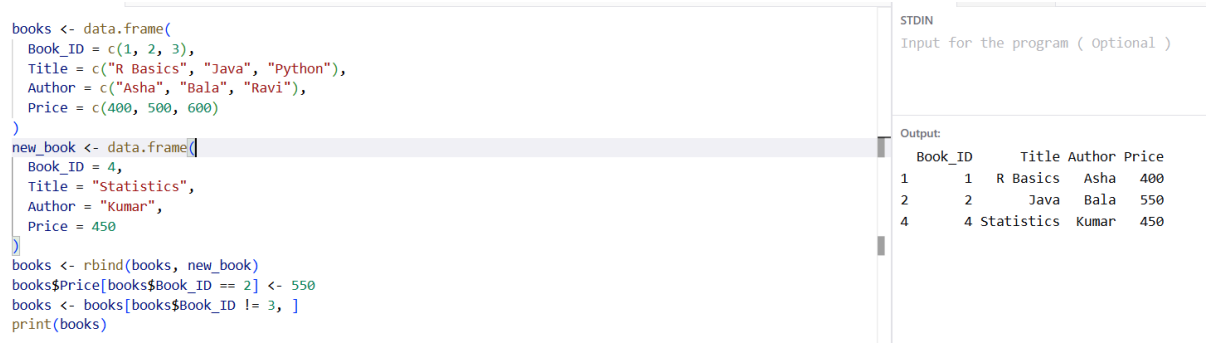
A library maintains a data frame containing Book ID, Title, Author, and Price. Develop an R program to add a new book, modify the price of an existing book, and delete a discontinued book. Analyze how these operations affect the structure and contents of the data frame.

CODE:

```
books <- data.frame(
  Book_ID = c(1, 2, 3),
  Title = c("R Basics", "Java", "Python"),
  Author = c("Asha", "Bala", "Ravi"),
  Price = c(400, 500, 600)
)
new_book <- data.frame(
  Book_ID = 4,
  Title = "Statistics",
  Author = "Kumar",
  Price = 450
)
books <- rbind(books, new_book)
books$Price[books$Book_ID == 2] <- 550
books <- books[books$Book_ID != 3, ]
```

print(books)

OUTPUT:



5. Data Frame Indexing and Filtering

A retail store stores Product ID, Product Name, Category, Price, and Stock Quantity in a data frame. Write an R program to extract selected columns, filter products priced above ₹1,000, and identify products with stock below 10 units. Analyze the differences between row indexing, column indexing, and logical filtering.

CODE:

products <- data.frame(

Product_ID = c(1, 2, 3, 4),

Product_Name = c("Laptop", "Mouse", "Phone", "Keyboard"),

Category = c("Electronics", "Accessories",

"Electronics", "Accessories"),

Price = c(50000, 500, 15000, 1200),

Stock = c(5, 25, 8, 6)

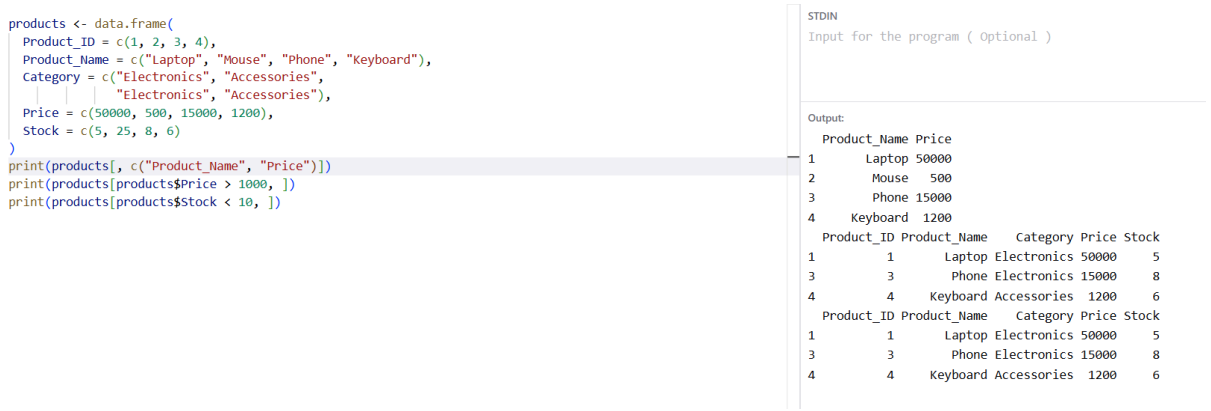
)

print(products[, c("Product_Name", "Price")])

print(products[products$Price > 1000,])

print(products[products$Stock < 10,])

OUTPUT:



6. Data Reshaping – Wide and Long Formats

A college records the marks of students in Mathematics, Statistics, and Programming using a wide-format data frame. Develop an R program to reshape the data into a long format and explain how the reshaped data supports subject-wise analysis. Compare the advantages of wide and long data formats.

CODE:

students <- data.frame(

  Name = c("Arun", "Priya", "Kavi"),

  Mathematics = c(80, 75, 90),

  Statistics = c(85, 70, 95),

  Programming = c(90, 80, 88)

)

long_data <- reshape(

  students,

```
  varying = c("Mathematics", "Statistics", "Programming"),
  v.names = "Marks",
  timevar = "Subject",
  times = c("Mathematics", "Statistics", "Programming"),
  idvar = "Name",
  direction = "long"
)

rownames(long_data) <- NULL

print(long_data)
```

OUTPUT:



```
students <- data.frame(
  Name = c("Arun", "Priya", "Kavi"),
  Mathematics = c(80, 75, 90),
  Statistics = c(85, 70, 95),
  Programming = c(90, 80, 88)
)

long_data <- reshape(
  students,
  varying = c("Mathematics", "Statistics", "Programming"),
  v.names = "Marks",
  timevar = "Subject",
  times = c("Mathematics", "Statistics", "Programming"),
  idvar = "Name",
  direction = "long"
)

rownames(long_data) <- NULL
print(long_data)
```

```
Input for the program ( Opti

Output:
   Name     Subject Marks
1  Arun Mathematics    80
2 Priya Mathematics    75
3  Kavi Mathematics    90
4  Arun  Statistics    85
5 Priya  Statistics    70
6  Kavi  Statistics    95
7  Arun Programming    90
8 Priya Programming    80
9  Kavi Programming    88
```

7. Melting Data Using R

A company records quarterly sales for four products across Q1, Q2, Q3, and Q4. Write an R program using the melt() function from the reshape2 package to convert the sales data into a long-format data frame. Analyze how melting simplifies product-wise and quarter-wise sales comparisons.

CODE:

```
library(reshape2)

sales <- data.frame(

  Product = c("Laptop", "Phone", "Tablet", "Monitor"),

  Q1 = c(50000, 40000, 20000, 15000),

  Q2 = c(55000, 45000, 22000, 18000),

  Q3 = c(60000, 48000, 25000, 20000),

  Q4 = c(65000, 50000, 28000, 22000)

)

long_sales <- melt(

  sales,

  id.vars = "Product",

  variable.name = "Quarter",

  value.name = "Sales"

)

print(long_sales)

print(aggregate(Sales ~ Product, long_sales, sum))

print(aggregate(Sales ~ Quarter, long_sales, sum))
```

OUTPUT:

```
> print(aggregate(Sales ~
  Quarter  Sales
1      Q1 125000
2      Q2 140000
3      Q3 153000
4      Q4 165000
>
```

8. Casting Data Using R

Using a long-format data frame containing Product, Quarter, and Sales, develop an R program using dcast() to convert the data back into a wide format. Generate a table showing products as rows and quarters as columns. Analyze how casting helps summarize and compare sales data.

CODE:

```
library(reshape2)

long_sales <- data.frame(

  Product = c("Laptop", "Laptop", "Phone", "Phone",

              "Tablet", "Tablet"),

  Quarter = c("Q1", "Q2", "Q1", "Q2", "Q1", "Q2"),

  Sales = c(50000, 55000, 40000, 45000, 20000, 22000)

)

wide_sales <- dcast(

  long_sales,

  Product ~ Quarter,

  value.var = "Sales"

)
```

```
print(wide_sales)
```

OUTPUT:

```
print(wide_sales)
Product    Q1    Q2
 Laptop 50000 55000
  Phone 40000 45000
 Tablet 20000 22000
'
```

9. Melting and Casting – Analytical Comparison

A hospital maintains patient records containing Patient ID, Department, and consultation expenses for three months. Design an R program to reshape the data using melting and casting techniques. Analyze how these transformations help compare monthly expenses across departments and identify the department with the highest expenditure.

CODE:

```
library(reshape2)

hospital <- data.frame(

  Patient_ID = c(101, 102, 103, 104),

  Department = c("Cardiology", "Neurology",

                 "Cardiology", "Orthopedics"),

  January = c(1500, 1200, 1800, 1000),

  February = c(1600, 1400, 1700, 1100),

  March = c(1800, 1300, 1900, 1200)

)

long_hospital <- melt(

  hospital,
```

```
  id.vars = c("Patient_ID", "Department"),
  variable.name = "Month",
  value.name = "Expense"
)
print(long_hospital)
monthly_expenses <- dcast(
  long_hospital,
  Department ~ Month,
  value.var = "Expense",
  fun.aggregate = sum
)
print(monthly_expenses)
department_totals <- aggregate(
  Expense ~ Department,
  data = long_hospital,
  FUN = sum
)
print(department_totals)
print(department_totals[
  which.max(department_totals$Expense), ])
```

OUTPUT:

```
print(department_totals)
 Department Expense
 Cardiology   10300
  Neurology    3900
Orthopedics    3300

# Department with highest expenditure
print(department_totals[
  which.max(department_totals$Expense), ])
Department Expense
Cardiology   10300
```

10. Merging Data Frames

Two data frames contain student details: one stores Student ID, Name, and Department, while the other stores Student ID, Marks, and Attendance. Write an R program to merge the data frames using Student ID as the common key. Analyze the differences between inner joins, left joins, right joins, and full joins, and justify which join is appropriate when all student records must be retained.

CODE:

```
student_details <- data.frame(
  Student_ID = c(101, 102, 103, 104),
  Name = c("Arun", "Priya", "Kavin", "Divya"),
  Department = c("IT", "CSE", "ECE", "IT")
)
student_marks <- data.frame(
  Student_ID = c(101, 102, 103, 105),
  Marks = c(85, 90, 78, 88),
  Attendance = c(90, 95, 80, 85)
```

```
)
inner_result <- merge(
  student_details, student_marks,
  by = "Student_ID"
)
left_result <- merge(
  student_details, student_marks,
  by = "Student_ID", all.x = TRUE
)
right_result <- merge(
  student_details, student_marks,
  by = "Student_ID", all.y = TRUE
)
full_result <- merge(
  student_details, student_marks,
  by = "Student_ID", all = TRUE
)
print(inner_result)
print(left_result)
print(right_result)
print(full_result)
```

OUTPUT:

```
>
> print(inner_result)
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    90         95
3        103 Kavin        ECE    78         80
> print(left_result)
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    90         95
3        103 Kavin        ECE    78         80
4        104 Divya         IT    NA         NA
> print(right_result)
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    90         95
3        103 Kavin        ECE    78         80
4        105  <NA>       <NA>    88         85
> print(full_result)
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    90         95
3        103 Kavin        ECE    78         80
4        104 Divya         IT    NA         NA
5        105  <NA>       <NA>    88         85
```

11. Merging Data with Missing Records

A company maintains separate data frames for employee details and monthly performance ratings. Some employees have no performance records, while some ratings refer to employees absent from the employee database. Develop an R program to merge these data frames using appropriate join operations. Analyze how unmatched records and missing values affect the final data set.

CODE:

```
employees <- data.frame(

  Employee_ID = c(201, 202, 203, 204),

  Name = c("Anu", "Bala", "Charan", "Devi")

)
```

```
performance <- data.frame(
  Employee_ID = c(201, 203, 205),
  Rating = c(4.5, 3.8, 4.2)
)
left_join <- merge(
  employees, performance,
  by = "Employee_ID",
  all.x = TRUE
)
right_join <- merge(
  employees, performance,
  by = "Employee_ID",
  all.y = TRUE
)
full_join <- merge(
  employees, performance,
  by = "Employee_ID",
  all = TRUE
)
print(left_join)
print(right_join)
```

print(full_join)

print(left_join[is.na(left_join$Rating),])

OUTPUT:

```
print(left_join)
Employee_ID   Name Rating
        201    Anu    4.5
        202   Bala     NA
        203 Charan    3.8
        204   Devi     NA
print(right_join)
Employee_ID   Name Rating
        201    Anu    4.5
        203 Charan    3.8
        205   <NA>    4.2
print(full_join)
Employee_ID   Name Rating
        201    Anu    4.5
        202   Bala     NA
        203 Charan    3.8
        204   Devi     NA
        205   <NA>    4.2

# Identify missing performance ratings
print(left_join[is.na(left_join$Rating), ])
Employee_ID Name Rating
        202 Bala     NA
        204 Devi     NA
```

12. Reading Data from Files

A researcher receives student performance data in a CSV file. Develop an R program to import the file using read.csv(), inspect its structure, identify missing values, and filter students whose average marks exceed 75. Analyze the importance of file paths, column names, data types, and missing-value handling during data import.

CODE:

students <- read.csv("C:/Users/ADMIN/Downloads/students.csv")

str(students)

summary(students)

```
colSums(is.na(students))

students <- students[!is.na(students$Average), ]

result <- students[students$Average > 75, ]

print(result)
```

OUTPUT:

```
> print(result)
   Name Average
1   Anu      82
3 Priya      91
5 Meena      76
>
```

13. Writing Data to Files

A sales analyst processes a data frame containing monthly revenue, expenses, and profit. Write an R program to calculate profit, select profitable months, and export the results using write.csv(). Analyze how exporting selected records supports reporting and data sharing, and explain the purpose of the row.names argument.

CODE:

```
sales <- data.frame(

  Month = c("Jan", "Feb", "Mar", "Apr"),

  Revenue = c(50000, 40000, 60000, 30000),

  Expenses = c(30000, 45000, 35000, 20000)

)

salesProfit <− salesRevenue - sales$Expenses
```

```
profitable <- sales[sales$Profit > 0, ]

write.csv(

  profitable,

  "profitable_months.csv",

  row.names = FALSE

)

print(profitable)
```

OUTPUT:

```
> print(profitable)
  Month Revenue Expenses Profit
1   Jan   50000    30000  20000
3   Mar   60000    35000  25000
4   Apr   30000    20000  10000
>
```

14. Editing and Updating File Data

A school stores student details in a CSV file. The administrator needs to correct student names, update marks, add new student records, and remove withdrawn students. Develop an R program to read the file, perform these modifications, and save the updated data to a new CSV file. Analyze the advantages of creating a separate output file instead of directly overwriting the original file.

CODE:

```
students <- data.frame(

  ID = c(101, 102, 103),

  Name = c("Anu", "Ravi", "Priya"),
```

```
  Marks = c(80, 75, 90),
  Status = c("Active", "Active", "Withdrawn")
)
students$Name[students$ID == 101] <- "Ananya"
students$Marks[students$ID == 102] <- 90
new_student <- data.frame(
  ID = 105,
  Name = "Karthik",
  Marks = 85,
  Status = "Active"
)
students <- rbind(students, new_student)
students <- students[students$Status != "Withdrawn", ]
write.csv(students, "updated_students.csv",
          row.names = FALSE)
print(students)
```

OUTPUT:

```
>
> print(students)
   ID    Name Marks Status
1 101  Ananya    80 Active
2 102    Ravi    90 Active
4 105 Karthik    85 Active
>
```

15. Integrated Data Frame Analysis

An e-commerce company maintains two CSV files: products.csv, containing Product ID, Product Name, and Category, and sales.csv, containing Product ID, Month, and Sales Amount.

CODE:

```
products <- read.csv("C:/Users/ADMIN/Downloads/products.csv")

sales <- read.csv("C:/Users/ADMIN/Downloads/sales.csv")

names(products)

names(sales)

names(products)[1] <- "ProductID"

names(sales)[1] <- "ProductID"

combined <- merge(products, sales,

                  by = "ProductID", all = TRUE)

print(combined)

total <- aggregate(

  sales$Sales.Amount,

  by = list(ProductID = sales$ProductID),

  FUN = sum

)

names(total)[2] <- "TotalSales"

print(total)
```

write.csv(combined, "combined_sales.csv",

row.names = FALSE)

OUTPUT:

```
[1] "Product.ID"    "Month"         "Sales.Amount"
>
> names(products)[1] <- "ProductID"
> names(sales)[1] <- "ProductID"
>
> combined <- merge(products, sales,
+                   by = "ProductID", all = TRUE)
>
> print(combined)
  ProductID Product.Name    Category Month Sales.Amount
1       101     Notebook  Stationery   Jan         1200
2       101     Notebook  Stationery   Feb         1400
3       102          Pen  Stationery   Jan          500
4       102          Pen  Stationery   Mar          650
5       103     Backpack Accessories   Feb         2500
6       104 Water Bottle Accessories   Mar         1800
>
> total <- aggregate(
+   sales$Sales.Amount,
+   by = list(ProductID = sales$ProductID),
+   FUN = sum
+ )
>
> names(total)[2] <- "TotalSales"
> print(total)
  ProductID TotalSales
1       101       2600
2       102       1150
3       103       2500
4       104       1800
>
> write.csv(combined, "combined_sales.csv",
+           row.names = FALSE)
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Logic & Vectorisation	30%	**(28–30 Points)** • Zero iterative loops where native vectorised operators (&, |, %%, ifelse) apply. • Code handles complete vectors in single computational cycles.	**(23–27 Points)** • Accurate control setups.	**(15–22 Points)** • Heavy reliance on nested loops (for, while) for basic transformations. • Calculation code scales poorly and experiences significant execution delay.	**(0–14 Points)** • Confuses vector and scalar logic tracks. • Triggers frequent length mismatch errors or script termination crashes.
Argument & Flow Integrity	20%	**(19–20 Points)** • Complete use of dynamic lazy evaluation properties for defaults. • Integrates clean early-exit guard clauses to save memory cycles.	**(15–18 Points)** • Clear parameter layout.	**(10–14 Points)** • Missing critical default parameters. • Logic fails if a user leaves out an optional calculation parameter during runtime.	**(0–9 Points)** • Broken function signatures. • Erroneously uses internal assignment operators (<-) to specify signature defaults.
Object Architecture & Returns	20%	**(19–20 Points)** • Combines multiple metrics	**(15–18 Points)** • Returns a	**(10–14 Points)** • Unpredictable return types	**(0–9 Points)** • Breaks isolated scope

		groups into a structured named list. • Completely relies on clean, implicit terminal line returns.	single clean data structure (matrix/data frame) properly.	across different internal calculation branches. • Output formats shift between vectors and matrices.	mechanics. • Updates external values using global modifications (<<-) instead of functional returns.
Edge Case Handling	15%	**(14–15 Points)** • Explicit validation blocks intercept empty inputs (numeric(0)) or missing data structures (NA, NaN, NULL) immediately.	**(11–13 Points)** • Code processes standard inputs cleanly. • Filters simple missing elements, but unmapped structures occasionally leak into operations.	**(8–10 Points)** • System crashes when an unexpected NA value is encountered inside strict logical comparisons.	**(0–7 Points)** • Complete lack of diagnostic checks. • Empty or missing data structures trigger fatal errors or infinite loops immediately.
Recursion & Memory Strategy	15%	**(14–15 Points)** • Implements rigid mathematical base cases. • Zero copy-on-write memory overhead achieved via	**(11–13 Points)** • Restricts recursive tasks to safe internal depth limits. • Keeps basic iteration	**(8–10 Points)** • Recursion paths lack solid boundary conditions. • Threatens system stack safety if evaluated beyond small data sets.	**(0–7 Points)** • Infinite recursion due to a missing base case block. • Triggers immediate stack overflow system crashes.

		pre-allocation or vectorised setups.	boundaries accurate.		